

TODD:

- Entscheidbarkeit
- Minimalautomat
- DEA/NEA/TM/NTM
- Beispiele:
 - Kann eine NTM in polynomieller Zeit entscheiden, ob ein Ausföhrtsel eine Lösung hat? → Satz von Rice / P Verifizierer / Reduktion
 - Ist Grammatik eindeutig? (FS_2026)5)
 - Parse-tree finden (FS_2022)5))
- Prüfungsaufgaben:
 - HS_2021)4)
 - FS_2022)4)
- Credits: Nina Grässli, Jannis Tschan, Jasmin

Begriff	Definition
Regulär	Es gibt einen DEA A, der L akzeptiert, also $L(A) = L$
Kontextfrei	L kann nur von einem ND PDA erkannt werden

VORGEHEN ZU AUFGABENTYPEN

Ist Sprache regulär?

Ja: DEA / NEA / RegEx

Nein: Pumping Lemma

Ist Sprache kontextfrei?

Ja: Stackautomat / CNF / BNF

Nein: Pumping Lemma (CFL)

Turing-Maschinen

Berechnungsgeschichte

Ist eine Sprache Turing-Vollständig?

TODD:

Ja: TM-Simulation

Nein: Halteproblem

Kann Problem von einem Computer gelöst werden?

TODD:

Reduktion auf Halteproblem

Kann ein Programm entscheiden, ob zulässige Inputs eine Eigenschaft erfüllen?

Satz von Rice

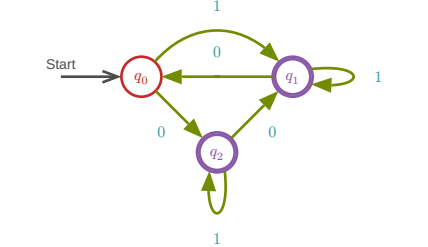
Kann eine nichtdeterministische Maschine ... entscheiden?

NP-Verifizierer

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

$A = (Q, \Sigma, \delta, q, F)$

- Zustände: $Q = \{q_0, q_1, \dots, q_n\}$
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$
- Startzustand: $q \in Q$
- Akzeptierzustände: $F \subset Q$



PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache nicht regulär ist

Vorgehen:

- Annahme: L ist regulär
- Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- Wähle ein Wort $w \in L$ mit $|w| \geq N$

- Die Definition muss N verwenden!
- Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- Auswirkung des Pumpens
 - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 1: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = 0^N 1^N$
- Unterteilung $w = xyz$
- Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt $xy^kz \notin L$ für $k \neq 1$, Widerspruch zum Pumping Lemma

Beispiel 2: $L =$ alle korrekte Integralausdrücke

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = \int_{a_1}^{b_1} \int_{a_2}^{b_2} \dots \int_{a_N}^{b_N} dx_N \dots dx_2 dx_1 \in L$
- Unterteile $w = xyz$, wobei $|y| \geq 1 \wedge |xy| \leq N \wedge xy^kz \in L$
- Pumpen: Der Teil xy enthält keine dx_k , beim pumpen nimmt also nur die Anzahl der Integralzeichen zu, so dass kein korrekter Integralausdruck stehen bleibt.
- Der Widerspruch zeigt, dass die Annahme, L sei regulär, nicht haltbar ist. Also ist L nicht regulär.

6 Schritte des Pumping-Lemma-Beweises: Annahme (A) 1 Punkt, Pumping Length (N) 1 Punkt, Wort (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Folgerung (F) 1 Punkt.

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

$A = (Q, \Sigma, \delta, q, F)$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$
- Startzustand: $q \in Q$
- Akzeptierzustände: $F \subset Q$

TODD:

Thompson-NEA

VERALLGEMEINERTER NEA (VNEA)

Begriff	Definition
Regulärer Ausdruck	Zeichenkette r zur Beschreibung einer regulären Sprache $L = L(r)$
Reguläre Operationen	$L(r_1) \cup L(r_2) = L(r_1 \mid r_2)$ $L(r_1)L(r_2) = L(r_1 r_2)$ $L(r_1)^* = L(r_1^*)$
Verallgemeinerter NEA	NEA _ε , dessen Übergänge mit regulären Ausdrücken beschriftet sind

PRIMITIVE REGULÄRE AUSDRÜCKE

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

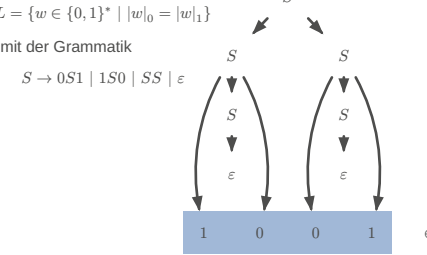
$L = L(r)$	r	NEA
$\emptyset$	$\emptyset$	Start 
$\{\epsilon\}$	$\{\epsilon\}$	Start 
$\{a\}$	a	Start 
$\{o, s, t\}$	$[ost]$	Start 
$\{a, b, \dots, s\}$	$[a-s]$	Start 
Σ	$.$	Start 

KONTEXTFREIE SPRACHEN (CFL)

$G = (V, \Sigma, R, S)$

- V: Variablen
- Σ: Terminalsymbole (Alphabet)
- R: Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$
- S: Startvariable

Parse Tree



PUMPING LEMMA (CFL)

Um zu beweisen, dass eine Sprache nicht kontextfrei ist

Voraussetzungen:

- $|vwy| > 0$
- $|vxy| \leq N$
- $\forall k \in \mathbb{N}. (uv^kxy^kz \in L)$

Beispiel 3: $L = \{w \in \{0,1\}^* \mid w = 0^k 1^0 10^k 10^l\}$

- Annahme: L ist kontextfrei
- Nach dem Pumping Lemma $\exists N \in \mathbb{N}$, Pumping Length
- Wähle das Wort $w = 0^N 10^N 10^N$
- Nach dem Pumping Lemma gibt es eine Aufteilung von w in fünf Teile $w = uvxyz$ derart, dass $|vxy| \leq N \wedge |vy| \geq 1$. Ausserdem ist jedes gepumpte Wort $uv^kxy^kz \in L$.
- Da sich die Anzahl der Einsen beim Pumpen nicht ändern darf, müssen v und y vollständig in einem Nullen-Block enthalten sein. Daher kann sich nur die Anzahl der Nullen in höchstens zwei Nullen-Blöcken ändern. Die beiden Blöcke müssen wegen $|vxy| \leq N$ ausserdem benachbart sein.

Zum ersten Nullen-Block gehört der dritte, der gleich viele Nullen enthalten muss, zum zweiten gehört der vierte. Wie auch immer die beiden Blöcke gewählt werden, ändert sich die Anzahl Nullen in den Blöcken, aber nicht in den zugehörigen Blöcken. Das gepumpte Wort kann also nicht mehr in L sein.

- Dieser Widerspruch zeigt, dass die Annahme, L sei kontextfrei, nicht haltbar ist. Also ist L nicht kontextfrei.

Pumping Lemma und Annahme L kontextfrei (PL) 1 Punkt, Pumping Length (N) 1 Punkt, Wahl eines Wortes (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Schlussfolgerung (S) 1 Punkt.

CHOMSKY-NORMALFORM (CNF)

- S kommt auf der rechten Seite nicht vor
- Jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$
- $S \rightarrow \epsilon$ ist erlaubt

UMWANDLUNG

1) Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)

2) ε-Regeln:

$$\left. \begin{matrix} A \rightarrow \epsilon \\ B \rightarrow AC \end{matrix} \right\} \Rightarrow \left\{ \begin{matrix} B \rightarrow AC \\ \rightarrow AC \end{matrix} \right.$$

3) Unit-Rules:

$$\left. \begin{matrix} A \rightarrow B \\ B \rightarrow CD \end{matrix} \right\} \Rightarrow \left\{ \begin{matrix} A \rightarrow CD \\ B \rightarrow CD \end{matrix} \right.$$

4) Verkettungen:

$$A \rightarrow u_1 u_2 \dots u_n$$
$$A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$$

falls u_i ein Terminalsymbol: $A_{i-1} \rightarrow U_i A_i, U_i \rightarrow u_i$

Beispiel 4: $S \rightarrow ASA \mid aB, A \rightarrow B \mid S, B \rightarrow b \mid \epsilon$

1) Startvariable

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \epsilon$$

2) ε-Regel

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a$$
$$A \rightarrow B \mid \epsilon \mid S$$
$$B \rightarrow b$$
$$\leadsto$$
$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b$$

3) Unit-Regel

$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$$
$$B \rightarrow b$$
$$\leadsto$$
$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$$
$$B \rightarrow b$$

4) Komplexe Regeln

$$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$
$$S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$
$$A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$$
$$A_1 \rightarrow SA$$
$$U \rightarrow a$$
$$B \rightarrow b$$

ABLEITUNGSDREIECK

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

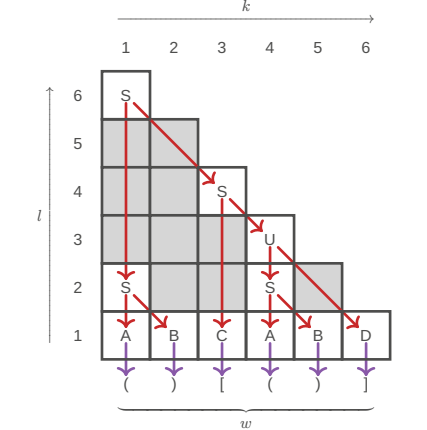
Beispiel

Parzen des Wortes ()()

$$S \rightarrow AB \mid CD \mid CU \mid SS$$
$$U \rightarrow SD$$
$$A \rightarrow ($$
$$B \rightarrow)$$
$$C \rightarrow [$$
$$D \rightarrow]$$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .



BNF

- Variablen: <variablen-name>
- Einzelne Zeichen: A
- Zeichenketten: 'BEISPIEL'
- Regeln: <variablen-name> ::= Ausdruck
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch |

STACKAUTOMAT (PDA)

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q: Zustände
- Σ: Eingabe-Alphabet
- Γ: Stack-Alphabet
- δ: $Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \rightarrow P(Q \times \Gamma_{\epsilon})$
- $q_0 \in Q$: Startzustand
- $F \subset Q$: Akzeptierzustände



a vom Input

b vom Stack entfernen (Bedingung)

c auf den Stack

STACKAUTOMAT STANDARDISIEREN

TODD:

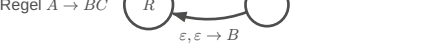
GRAMMATIK ABLESEN

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P, der L akzeptiert, $L = L(P)$.

Start



Regel $A \rightarrow BC$



Regel $A \rightarrow a$



Regel $S \rightarrow \epsilon$



$\forall a \in \Sigma$



TODD:

$$L = \{0^n 1^n \mid n \geq 0\}, w = 0011$$

q_0	0	0	1	1
x	q_1	0	1	1
x	0	q_1	1	1
x	0	x	q_2	1
x	0	x	1	q_2
x	0	x	q_3	1
x	0	q_3	x	1
x	q_3	0	x	1
q_3	x	0	x	1
x	x	0	x	1
q_0	x	0	x	1
x	q_0	0	x	1
x	x	q_1	x	1
x	x	x	q_1	1
x	x	x	x	q_2
x	x	x	q_3	x
x	x	q_3	x	x
q_3	x	x	x	x
q_3	x	x	x	x
q_0	x	x	x	x
x	q_0	x	x	x
x	x	q_0	x	x
x	x	x	q_0	x
x	x	x	x	q_{accept}

Simulierbar in Zeit $O(t(n))$

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

Die Eigenschaft P Turing-erkennbarer Sprachen heit **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

	Schritt	Aufwand
1	Jedes Feld: Zeichen kommt auf der Zeile nicht mehr vor	$O(n^4 \cdot n^2)$

2	Jedes Feld: Zeichen kommt in der Spalte nicht mehr vor	$O(n^4 \cdot n^2)$
3	Jedes Feld: Zeichen kommt im Unterquadrat nicht vor	$O(n^4 \cdot n^2)$
Total		$O(n^6)$

NURIKABE

Entscheidbar? Ja: $n = uv$ = Anzahl Felder. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.

Zertifikat: Liste der schwarzen Felder

Vorgehen:

	Schritt	Aufwand
1	Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^3)$
2	Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3	Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4	Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5	Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
Total		$O(n^3)$

DAMEN

TODO:

REDUKTION

Polynomielle Reduktion: $A \leq_P B$. Lies: A ist polynomiell leichter entscheidbar als B .

TODO:

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

Zu beachten (Punkteverteilung):

- Lösungsprinzip der Reduktion erwähnen (1P)
- Auswahl eines geeigneten Vergleichsproblems (1P)
- Reduktionsabbildung (*P)
- Schlussfolgerung: NP-Vollständig und somit nicht effizient lösbar (1P)

TODO:

TURING-VOLLSTÄNDIG

Eine Sprache A heisst eine **Programmiersprache**, wenn es eine Abbildung $c : A \mapsto \Sigma^*$ gibt, wobei $c(w)$ ein Programm für eine universelle Turing-Maschine ist.

Eine Programmiersprache heisst **Turing-vollständig**, wenn in ihr jede beliebige Turing-Maschine simuliert werden kann.

KARP-KATALOG

PROBLEME MIT k ZAHLEN

VERTEX-COLORING

EXACT-CLIQUE-COVER

VERTEX-COVER

SET-COVERING

STEINER-TREE

FEEDBACK-NODE-SET

GRAPH-ÜBERDECKUNGEN

VERTEX-COLORING

CLIQUE-COVER

EXACT-CLIQUE-COVER

K-CLIQUE

VERTEX-COVER

SET-COVERING

STEINER-TREE

FEEDBACK-NODE-SET

GRAPH-ÜBERDECKUNGEN

VERTEX-COLORING

CLIQUE-COVER

EXACT-CLIQUE-COVER

K-CLIQUE

VERTEX-COVER

MENGEN

Handeln von einer Familie $(S_i)_{i \in I}$ von nichtleeren endlichen Mengen. In diesen Problemen geht es um die Existenz einer Teilfamilie mit bestimmten Eigenschaften.

SET-PACKING

Wie viele der Mengen S_i kann man in U hineinpacken, ohne dass sie sich überlappen? Gibt es k Teilmengen in S , welche disjunkt sind und die Menge U abdecken?

SET-COVERING

Gibt es eine Unterfamilie bestehend aus k Mengen, die die gleiche Vereinigung U hat? Kann man k Teilmengen bilden, welche die Menge S komplett abdecken?

CLIQUE-COVER

Gibt es k Cliques so, dass jede Ecke in genau einer der Cliques ist?

EXACT-CLIQUE-COVER

Wie CLIQUE-COVER, nur müssen die Mengen disjunkt sein.

K-CLIQUE

Wie CLIQUE-COVER, nur ist k bekannt.

VERTEX-COVER

Gibt es eine Menge W von k Knoten derart, dass jede Kante von G einen Endpunkt in W hat?

AUFTEILUNG IN ZWEI MENGEN

PARTITION

Gibt es eine Unterteilung einer Liste S von ganzen Zahlen in zwei disjunkte Listen mit gleicher Summe?

MAX-CUT

Gegeben ein Graph $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$ der Kanten und einer ganzen Zahl $k \in \mathbb{Z}$, gibt es eine Aufteilung der Knotenmenge $V = V_1 \cup V_2$ in zwei disjunkte Teilmengen, so dass das Gewicht der Kanten, die V_1 und V_2 verbinden, mindestens w ist:

STEINER-TREE

Gibt es zu einem Graphen $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$, einer Teilmenge $R \subset V$ der Knoten und einer Zahl k einen Teilbaum $T = (V_1, E_1) \subset G$ des Graphen, der alle Knoten von R enthält und dessen Kantengewicht $\sum_{e \in E_1} \varphi(e) \leq k$ ist?

SEQUENCING

Die Komponenten der drei Vektoren

PFAD IN GRAPHEN

Ein hamiltonscher Pfad in einem Graphen $G = (V, E)$ ist ein Pfad, der jeden Knoten des Graphen genau einmal besucht.

HAMPATH

Gibt es einen gerichteten hamiltonschen Pfad im gerichteten Graphen G ?

HAMCIRCUIT

Wie HAMPATH, aber zusätzlich muss der Pfad geschlossen sein

UHAMPATH

Wie HAMPATH, aber ungerichtet

UHAMCIRCUIT

Wie UHAMPATH, aber zusätzlich muss der Pfad geschlossen sein

GERICHTETE GRAPHEN

Es handelt sich um die Eigenschaften von Zyklen in gerichteten Graphen $G = (V, E)$. Die Kantenmenge E besteht aus gerichteten Kanten (a, e) mit $a, e \in V$. Ein gerichteter Zyklus in G ist eine Folge von Knoten $a_0, \dots, a_n = a_0$, die jeweils Enden von Kanten sind, also $(a_i, a_{i+1}) \in E, i = 0, \dots, n-1$

FEEDBACK-NODE-SET

Gibt es zu einem gerichteten Graphen G und einer Zahl k eine Menge $W \subset V$ von k Knoten derart, dass jeder geschlossene Zyklus durch einen dieser Knoten geht?

FEEDBACK-ARC-SET

Gibt es zu einem gerichteten Graphen G und einer Zahl k eine Menge $F \subset E$ von k Kanten derart, dass jeder geschlossene Zyklus eine dieser Kanten enthält?

LOGIK

SAT

Gegeben einer aussagenlogischen Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?

3SAT

Wie SAT, nur hat jede Klausel der Normalform maximal 3 Literale.

WEITERE

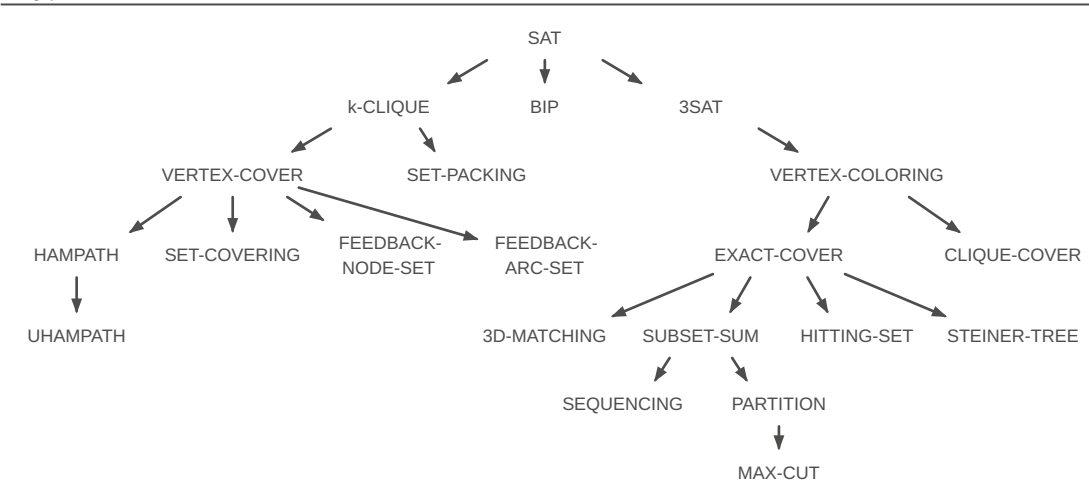
SUBSET-SUM

Gegeben einer Liste S von natürlichen Zahlen $s_i \in \mathbb{N}$ und einer natürlichen Zahl $t \in \mathbb{N}$, ist es möglich eine Teilmenge $T \subset S$ auszuwählen, sodass die Summe $t = \sum_{s \in T} s$ haben?

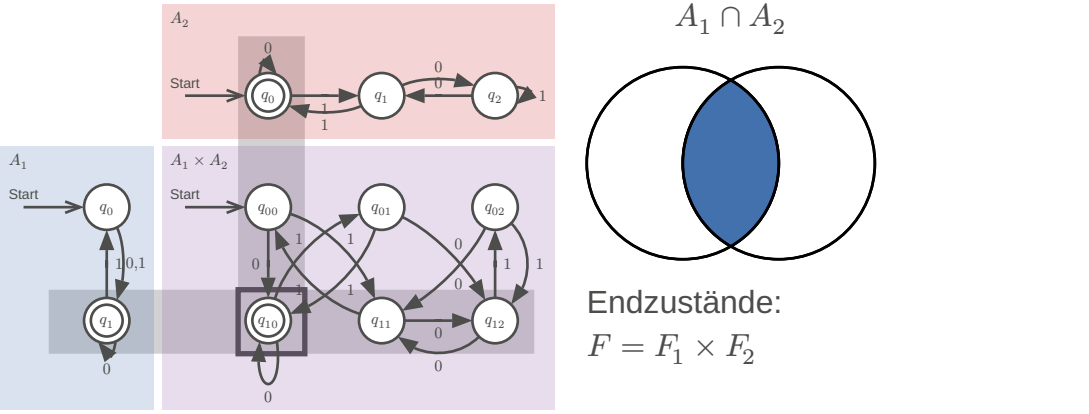
3D-MATCHING

Gegeben sind drei endliche Mengen X, Y und Z mit gleich vielen $n = |X| = |Y| = |Z|$ Elementen und $T \subset X \times Y \times Z$. Gibt es eine n -elementige Teilmenge $M \subset T$ derart, dass keine zwei Tripel von M in einer Koordinate übereinstimmen?

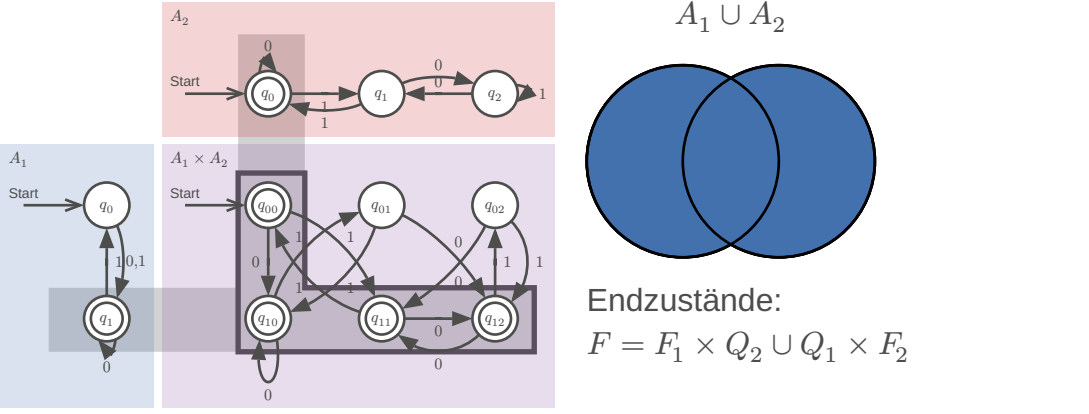
Diagramme



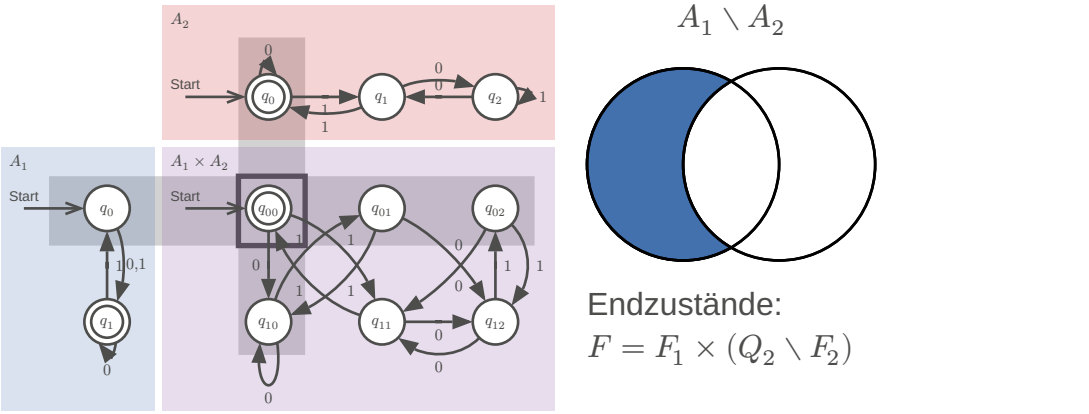
MENGEN
SCHNITTMENGE $L(A_1) \cap L(A_2)$



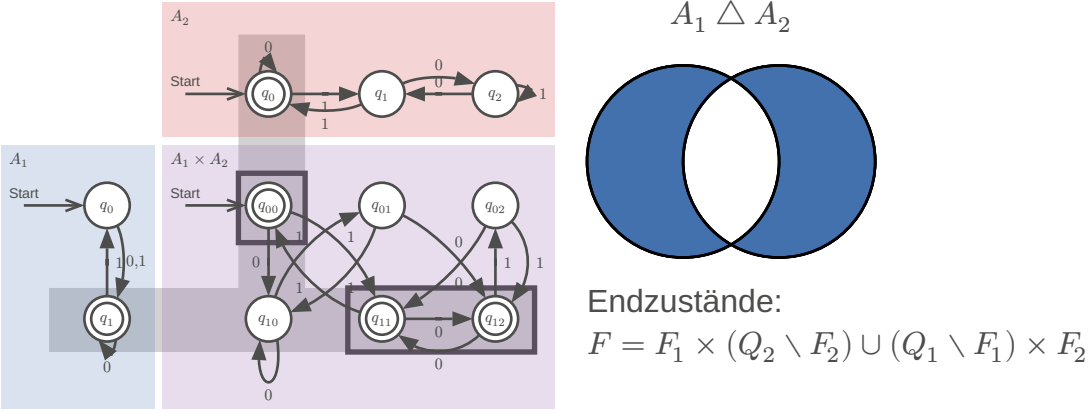
VEREINIGUNGSMENGE $L(A_1) \cup L(A_2)$



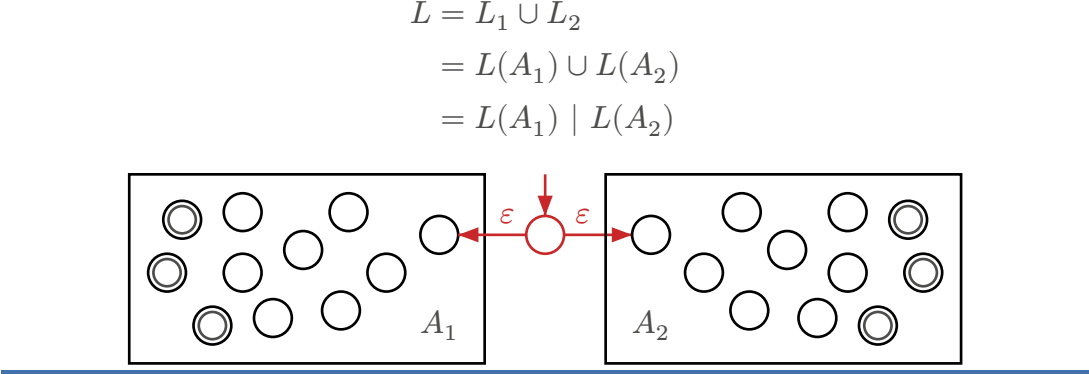
DIFFERENZMENGE $L(A_1) \setminus L(A_2)$



SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$

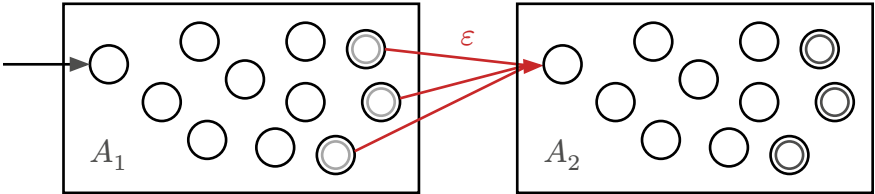


REGULÄRE AUSDRÜCKE
ALTERNATIVE



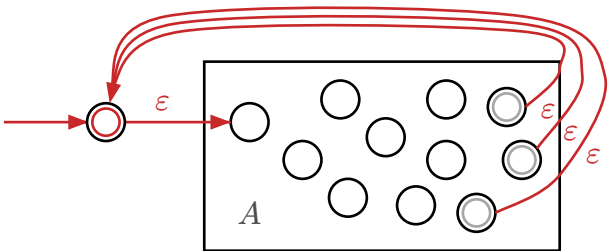
VERKETTUNG

$$L = L_1 L_2$$
$$= L(A_1) L(A_2)$$
$$= \{w_1 w_2 \mid w_i \in L_i\}$$



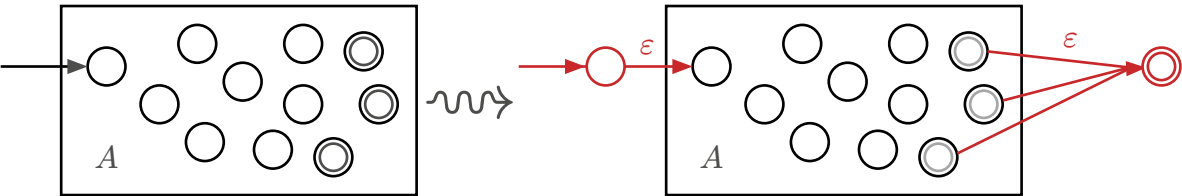
*-OPERATION

$$L^* = \{\epsilon\} \cup L \cup L^2 \cup \dots$$
$$= \bigcup_{k=0}^{\infty} L^k$$



VNEA ZU REGEX

Keine Übergänge nach q_0 und nur ein Akzeptierzustand



Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruck r von A

